

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE CODE: CSA1402 COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 20 Total Marks:100 Annexure A

SHORT ANSWER TEST

Code of Conduct:

I V. Vamsidhar Reddy -192365029 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary

action. Signature of the student: Vamsi

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Explanation	25%				
Application	20%				
Organization	15%				
Accuracy	10%				
Clarity	5%				

Faculty Incharge

Annexure A

SHORT ANSWER TEST

1. Construct language for the following Regular Expression:2

$$(a|b)^*a(a|b)(a|b)$$

2. a) Illustrate the predictive parser for the following grammar. 4

$$S \rightarrow (L) \mid a$$

$$L \rightarrow L, S \mid S$$

3. Analyse, is it possible, by modifying the grammar in any way to construct predictive parser for the language of

$$S \rightarrow SS^+ \mid SS^* \mid a$$

String: "aa+a" 4

4. Eliminate left recursion from the grammar 2

5. The following grammar contains common prefixes, making it unsuitable for predictive parsing.

$$A \rightarrow abcD \mid abcE \mid f$$

Apply left factoring to remove the common prefixes and rewrite the grammar.5

6. Consider the following statement:10

Show the representation of this statement after each phase of the compiler and explain.

7. Consider the following C code fragment:4

```
sum = 0;
```

```
for (i = 1; i < 5; i++)
```

```
    sum = sum + i;
```

Show the representation of the above code after each phase of the

compiler.2 8. Consider the following grammar.

Calculate FIRST and FOLLOW, Obtain the Predictive Parsing Table.

not. Show the parser moves for the input string **acdb**.

9. Write a simple Lex program to recognize and count identifiers in a given input.2

10. Write the rule to Eliminate Left Recursion and Left Factoring.2 11. What are the actions performed by a shift-reduce parser? Explain each action with a suitable example.3

12. Consider the following grammar:

Find the leftmost derivation and construct the parse tree for the input string pqqqr.3

13. Consider Grammar: $E \rightarrow E+T \mid T$ $T \rightarrow T^*F \mid F$ $F \rightarrow (E) \mid id$

Eliminate left recursion and find FIRST and FOLLOW 5

14. **For the following grammar, determine whether the string **abcc** is accepted using shift-reduce parsing. Show all shift and reduce operations.**

10

15. For the following grammar, verify whether the string **abbc** is valid using shift reduce parsing.10

Show the complete sequence of stack, input, and parser actions.

16. Consider the following grammar:10

- Construct the canonical collection of LR(0) items.
- Construct the SLR(1) parsing table.

17. Consider the following grammar.

Obtain the Predictive Parsing Table. 5

18. Do left factoring in the following grammar: 3

A-> aAB|aA|a

B-> bB|b

19. Construct the LL(1) Parsing Table for the following grammars.7

20. Construct the LL(1) Parsing Table for the following grammars.7

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understandi ng of Concepts	25%	Demonstrat es thorough, accurate understandi ng of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understandi ng; major misconce pti ons
Explanation	25%	Detailed, well explained answers with relevant	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation

examples

Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No applicati on or incorrect analysis
-------------	-----	--	--	---	---

Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Q. Predictive Parser for the following grammar.

$$S \rightarrow (L) | a$$

$$L \rightarrow L, S | S$$

63
1000

Step 1: Eliminate the Left recursion.

$$S \rightarrow (L) | a$$

$$L \rightarrow L, S | S$$

$$L \rightarrow SL'$$

$$L' \rightarrow , SL' | \epsilon$$

$$S \rightarrow (L) | a$$

$$L \rightarrow SL'$$

$$L' \rightarrow , SL' | \epsilon$$

Step 2: find first() and follow()

$$\text{first}(S) = \{ (, a \}$$

$$\text{first}(L) = \{ (, a \}$$

$$\text{first}(L') = \{ , , \epsilon \}$$

$$\text{follow}(S) = \{ \$, (, a \}$$

$$\text{follow}(L) = \{) \}$$

$$\text{follow}(L') = \{), \$ \}$$

	(	a	,	)
S	$S \rightarrow (L) a$	$S \rightarrow (L) a$		
L	$L \rightarrow SL'$	$L \rightarrow SL'$		
L'			$L' \rightarrow , SL'$	$L' \rightarrow \epsilon$

③ Predictive Parser

$$S \rightarrow SS + / SS * / a$$

String: "aa + a"

eliminate left recursion

$$S \rightarrow aS'$$

$$S' \rightarrow +SS' / *SS' / \epsilon$$

first() & follow()

$$\text{first}(S) = \{a\}$$

$$\text{first}(S') = \{+, *, \epsilon\}$$

$$\text{follow}(S) = \{\$, +, *\}$$

$$\text{follow}(S') = \{\epsilon\}$$

Stack

\$

Input

Output

4) Eliminate Left recursion

$$S \rightarrow Sa | Sb | c$$

$$S \rightarrow cS'$$

$$S' \rightarrow aS' | bS' | \epsilon$$



5) ~~Predictive Parsing~~

$$A \rightarrow abeD | abceE | f$$

~~eliminate Left recursion~~

$$A \rightarrow abeD | abceE | f \rightarrow \text{No left recursion}$$

$$\text{first follow}(A) = \{\underline{a}, \underline{b}, \underline{e}, a, f\}$$

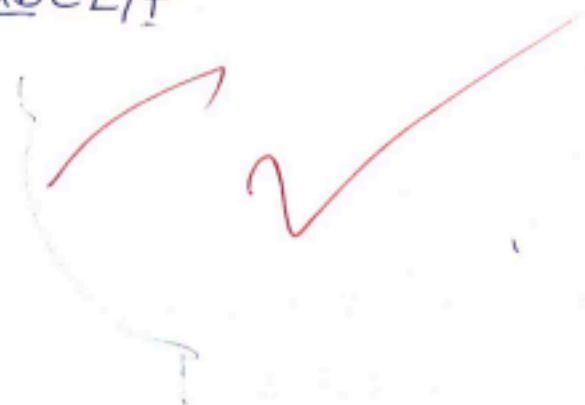
$$\text{follow}(A) = \{ \}$$

5) ~~→ Apply Left factoring :~~

$$A \rightarrow \underline{a}beD | \underline{a}bceE | f$$

$$A \rightarrow ab'A' | f$$

$$A' \rightarrow eD | cE$$



③ ⑥ Consider :-

$$\text{Degree-f} = \text{Degree-C} * 1.8 + 32$$

① Syntax Analysis

Syntax Analysis makes the equation into trees.



⑧ Consider the grammar

$$\begin{aligned} S &\rightarrow aABb, A \rightarrow \epsilon \\ A &\rightarrow c/\epsilon \\ B &\rightarrow d/\epsilon \end{aligned}$$

① Apply Left recursion

$$S \rightarrow aABb$$

$$A \rightarrow c/\epsilon$$

$$B \rightarrow d/\epsilon$$

No Left recursion

② first() and follow()

$$\text{first}(S) = \{a\}$$

$$\text{first}(A) = \{c, \epsilon\}$$

$$\text{first}(B) = \{d, \epsilon\}$$

$$\text{follow}(S) = \{\$ \}$$

$$\text{follow}(A) = \{d, \$ \}$$

$$\text{follow}(B) = \{b, \$ \}$$

	a	b	c	d	\$
S	$S \rightarrow aABb$				
A		$A \rightarrow \epsilon$	$A \rightarrow c$	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$
B		$B \rightarrow \epsilon$		$B \rightarrow d$	$B \rightarrow \epsilon$

Stack Implementation

i/p string : acdb

Stack	action	output
\$ s	acdb\$	
\$ bBAp	acdb\$	$S \rightarrow aABb$
\$ bBc	acdb\$	$A \rightarrow c$
\$ b	acdb\$	$B \rightarrow d$
\$	acdb\$	$B \rightarrow \epsilon$

⑨ Rules for Left recursion

if $A \rightarrow \alpha A / B$

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' / \epsilon$

where, α, β are terminals
and A is non terminal

* If equation produces some non terminal as S
then we need to remove the non terminal, by
introducing $\{A'\}$

Rule for Left factoring

$$A \rightarrow \alpha B_1 / \alpha B_2 / \dots / \alpha_1 / \alpha_2 \dots$$

then,

$$A \rightarrow \alpha A' / \alpha_1 / \alpha_2 \dots$$

$$A' \rightarrow B_1 / B_2 / \dots$$

L

① Shift reduce parsing

① if the operator in stack is less than or equal operator in input then perform shift.

② If the operator ~~have~~ in the stack have any the Reduce ~~it~~.

example

$$S \rightarrow S + S$$

$$S \rightarrow S - S$$

$$\underline{S \rightarrow (S) (S)}$$

$$S \rightarrow id$$

input string = id

Stack

\$

\$ id

\$ +

\$ S +

action

id + id - (id - id)

+ id - (id - id)

+ id - (id - id)

id - (id - id)

act

shift

Reduce

shift

shift

\$ S + id

$-(id - id)\$$

Reduc
S

\$ S + S

$-(id - id)\$$

Redu

\$ S

$-(id - id)\$$

S

sh

\$ S -

$(id - id)\$$

sh

\$ S - (

$id - id)\$$

sh

\$ S - (id

$- id)\$$

Red

\$ S - (S

$- id)\$$

sh

\$ S - (S -

$id)\$$

S

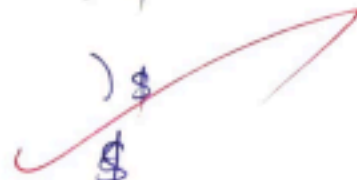
\$ S - (S - id

)\$

Pa

\$ S - (S - S)

)\$



\$ S - (S - S)

\$

Re

\$ S - (S)

\$

Re

\$ S - S

\$

Re

\$ S

\$

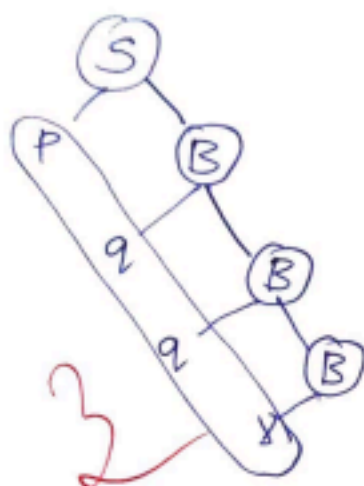
Acce

3

⑫ $S \rightarrow PB$

$B \rightarrow qB/y$

Construct parse tree for Pqqy



⑬ $E \rightarrow E+T/T$

$T \rightarrow T*F/F$

$F \rightarrow (E)/id$

* Eliminate Left recursion

$E \rightarrow TE'$

$E' \rightarrow +TE'/\epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT'/\epsilon$

$F \rightarrow (E)/id$

* first() & follow()

$$\text{first}(E) = \{ C, id \}$$

$$\text{first}(E') = \{ +, \epsilon \}$$

$$\text{first}(T) = \{ C, id \}$$

$$\text{first}(T') = \{ *, \epsilon \}$$

$$\text{first}(F) = \{ C, id \}$$

$$\text{follow}(E) = \{ \$, \rangle \}$$

$$\text{follow}(E') = \{ \$, \rangle \}$$

$$\text{follow}(T) = \{ +, \$, \rangle \}$$

$$\text{follow}(T') = \{ +, \$, \rangle \}$$

$$\text{follow}(F) = \{ +, \$, *, \rangle \}$$

(14)

$$S \rightarrow AB$$

$$A \rightarrow aA | a$$

$$B \rightarrow CB | C$$

input String: abcc

<u>Stack</u>	<u>Input</u>	<u>Action</u>
\$	abcc\$	shift
\$a	bcc\$	Reduce
\$A	bcc\$	shift
\$Ab	cc\$	shift
\$Abc	c\$	Reduce
\$AB	c\$	shift
\$AbBC	\$	Reduce
\$ABBB	\$	Accept

(15) Shift Reduce Parsing

$S \rightarrow aAB$

$A \rightarrow bA/b$

$B \rightarrow c$

Input string : abbc

<u>Stack</u>	<u>Input</u>	<u>Action</u>
\$	abbc\$	shift
\$a	bbc\$	shift
\$ab	bc\$	Reduce

\$aA

\$aAb

\$aAA

\$aAAC

\$aAAB

Q

bc\$

c\$

c\$

\$

\$

~~0~~

shift

Reduce

shift

Accept

Accept

16

$S' \rightarrow S$

$S \rightarrow aAB$

$A \rightarrow bA|c$

$B \rightarrow d$

$\rightarrow (I_0, S) = I_1$

$S' \rightarrow S \cdot$

$\rightarrow (I_0, a) = I_2$

$S \rightarrow a \cdot AB$

$\rightarrow (I_0, b) = I_3$

$A \rightarrow b \cdot A|c$

$\rightarrow (I_0, d) = I_4$

$B \rightarrow d \cdot$

$\rightarrow (I_0, c) = I_5$
 $A \rightarrow c \cdot$

Introduce $\cdot$

$S' \rightarrow \cdot S$

$S \rightarrow \cdot aAB$

$A \rightarrow \cdot bA|c$

$B \rightarrow \cdot d$

$\rightarrow (I_2, A) = I_6$

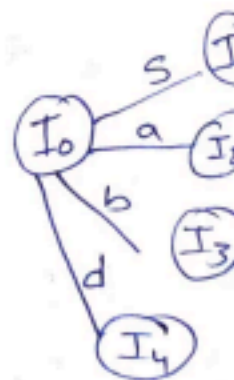
$S \rightarrow aA \cdot B$

$\rightarrow (I_3, A) = I_7$

$A \rightarrow bA \cdot$

$\rightarrow (I_6, B) = I_8$

$S \rightarrow aAB \cdot$



SLR Parsing Table :

S.No	a	b	c	d	A	B
1	S_2	S_3	S_5	S_4		
2						
3						
4						
5						
6						
7	r_2	r_3	r_4	r_5		
8		r_3				

(18)

Left factoring

$$A \rightarrow \underline{a}AB / \underline{a}A / \underline{a}$$

$$B \rightarrow bB / b$$

$$A \rightarrow aA'$$

$$A' \rightarrow AB / A / \epsilon$$

$$B \rightarrow bB'$$

$$B' \rightarrow B / \epsilon$$

$$S \rightarrow i E t s s' | a$$

$$s' \rightarrow e s / \epsilon$$

$$E \rightarrow b$$

→ No Elimination of Left Recursion

→ first() & follow()

$$\text{first}(S) = \{i, a\}$$

$$\text{first}(s') = \{e, \epsilon\}$$

$$\text{first}(E) = \{b\}$$

$$\text{follow}(S) = \{\$, e, a\}$$

$$\text{follow}(s') = \{\$, e\}$$

$$\text{follow}(E) = \{t, \$, e, a, \}$$

✓ 4

	i	t	a	e	b	\$
S	$S \rightarrow i E t s s'$	.	$S \rightarrow i E t s s'$			
s'				$s' \rightarrow e s / \epsilon$		$s' \rightarrow \epsilon$
E					$E \rightarrow b$	

(19)

$$S \rightarrow AEC$$

$$A \rightarrow a/\epsilon$$

$$B \rightarrow x/\epsilon$$

$$C \rightarrow b/\epsilon$$

$$\text{first}(S) = \{a, \epsilon\}$$

$$\text{first}(A) = \{a, \epsilon\}$$

$$\text{first}(B) = \{x, \epsilon\}$$

$$\text{first}(C) = \{b, \epsilon\}$$

$$\text{follow}(S) = \{\$, a\}$$

$$\text{follow}(A) = \{\$, a\}$$

$$\text{follow}(B) = \{\$, a, x\}$$

$$\text{follow}(C) = \{\$, a, x, b\}$$

2